EXPERIMENT 15 – BPTT FLOW AND GRADIENT ANALYSIS

1. Title

BPTT Flow and Gradient Analysis

2. Aim

To analyse Backpropagation Through Time (BPTT) in a Recurrent Neural Network using stock closing-price data. To calculate and visualise gradient magnitudes at different time steps for sequence lengths of 10, 50 and 100 days, compare tanh and ReLU activation functions, and study the vanishing and exploding gradient problems.

3. Mathematical Formulae

1. RNN Hidden State

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Where:

- x_t = Input at time step t
- h_t = Current hidden state
- h_{t-1} = Previous hidden state
- W_{xh} = Input-to-hidden weights
- W_{hh} = Recurrent weights
- b_h = Bias

2. Tanh Activation Function

$$f(x) = \tanh(x)$$

3. ReLU Activation Function

$$f(x) = \max(0, x)$$

4. Backpropagation Through Time

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

Where:

- L = Loss function
- T = Final time step
- t = Current time step

5. Gradient Norm

$$G_t = \left\| \frac{\partial L}{\partial h_t} \right\|_2$$

6. Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

7. Gradient Clipping Concept

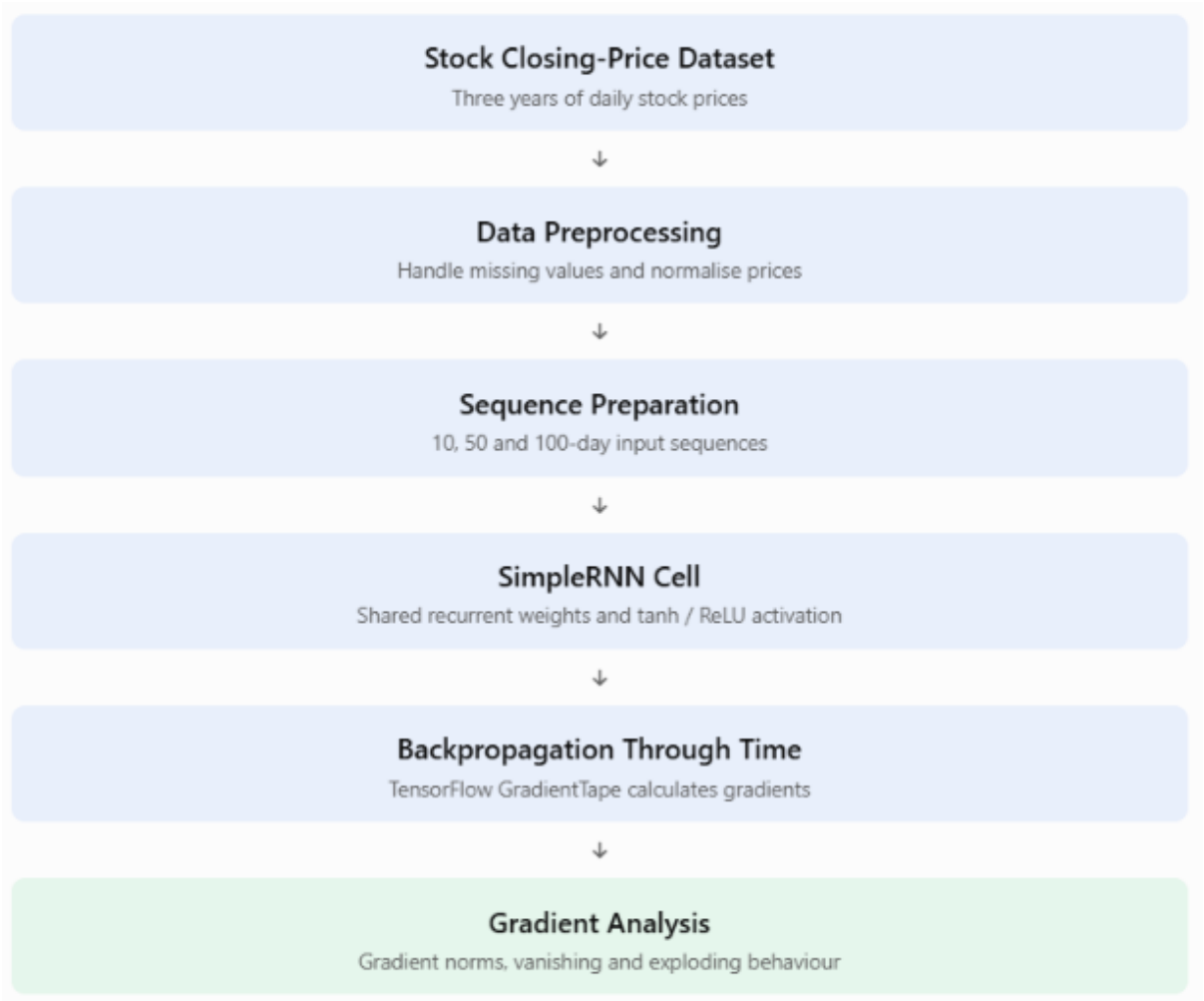
$$g_{clipped} = g \times \min\left(1, \frac{c}{\|g\|}\right)$$

Where c is the maximum permitted gradient norm.

Interpretation:

- Vanishing gradients: gradient norm becomes extremely small as it moves towards earlier time steps.
- Exploding gradients: gradient norm becomes extremely large during backward propagation.

4. System Architecture



5. Algorithm

1. Import TensorFlow, NumPy, Pandas and Matplotlib.
2. Generate or load three years of daily stock closing prices.
3. Handle missing values and normalise the price data.
4. Prepare input sequences with lengths of 10, 50 and 100 days.
5. Build a SimpleRNN cell with shared recurrent weights.
6. Perform forward propagation through each time step.
7. Calculate the prediction loss using Mean Squared Error.
8. Use TensorFlow GradientTape to calculate gradients with respect to each hidden state.
9. Calculate the gradient norm at every time step.
10. Repeat the analysis using tanh and ReLU activation functions.
11. Plot gradient norms against time steps for all sequence lengths.
12. Compare the plots and identify vanishing or exploding gradient behaviour.

6. Python Program

```
import numpy as np

import pandas as pd

import tensorflow as tf

import matplotlib.pyplot as plt


from sklearn.preprocessing import MinMaxScaler


# 1. Generate three years of daily stock closing prices

np.random.seed(42)


days = 3 * 365


dates = pd.date_range(

   start="2023-01-01",

   periods=days,
```

```
    freq="D"
)

trend = np.linspace(100, 150, days)

seasonal = 5 * np.sin(
    2 * np.pi * np.arange(days) / 30
)

noise = np.random.normal(0, 2, days)

prices = trend + seasonal + noise

df = pd.DataFrame({
    "Date": dates,
    "Close": prices
})

print("Stock Price Dataset:")
print(df.head())

# 2. Handle missing values
df["Close"] = df["Close"].interpolate()

# 3. Normalise stock prices
scaler = MinMaxScaler()

scaled_prices = scaler.fit_transform(
    df[["Close"]]
)
```

```
# 4. Define sequence lengths
sequence_lengths = [10, 50, 100]

activations = ["tanh", "relu"]

gradient_results = {}

# 5. Function to calculate gradient norms
def calculate_gradients(sequence_length, activation):

  # Prepare input sequence
  X = scaled_prices[:sequence_length]

  X = tf.convert_to_tensor(
    X.reshape(1, sequence_length, 1),
    dtype=tf.float32
  )

  target = tf.constant(
    [[scaled_prices[sequence_length][0]]],
    dtype=tf.float32
  )

  # Build SimpleRNN cell
  cell = tf.keras.layers.SimpleRNNCell(
    units=32,
    activation=activation
  )

  # Build output layer
  output_layer = tf.keras.layers.Dense(1)
```

```
# Initialise cell and output layer
cell(
  tf.zeros((1, 1)),
  [tf.zeros((1, 32))]
)

output_layer(
  tf.zeros((1, 32))
)

hidden_states = []

# 6. Forward and backward propagation
with tf.GradientTape() as tape:

  hidden_state = tf.zeros((1, 32))

  # Process each time step
  for t in range(sequence_length):

    current_input = X[:, t, :]

    output, states = cell(
      current_input,
      [hidden_state]
    )

    hidden_state = states[0]

    # Watch each hidden state
```

```
            tape.watch(hidden_state)

            hidden_states.append(hidden_state)

        # Final prediction
        prediction = output_layer(hidden_state)

        # Calculate loss
        loss = tf.reduce_mean(
            tf.square(target - prediction)
        )

    # 7. Calculate gradients for all hidden states
    gradients = tape.gradient(
        loss,
        hidden_states
    )

    # 8. Calculate gradient norms
    gradient_norms = []

    for gradient in gradients:

        norm = tf.norm(gradient).numpy()

        gradient_norms.append(norm)

    return gradient_norms, loss.numpy()

# 9. Analyse different sequence lengths and activations
for activation in activations:
```

```
    for length in sequence_lengths:

        gradients, loss = calculate_gradients(
            length,
            activation
        )

        gradient_results[
            (activation, length)
        ] = gradients

        print("\nActivation:", activation)
        print("Sequence Length:", length)
        print("Loss:", loss)

        print("Initial Gradient Norm:", gradients[0])
        print("Final Gradient Norm:", gradients[-1])

# 10. Plot gradient norms for tanh
plt.figure(figsize=(10, 5))

for length in sequence_lengths:

    plt.plot(
        range(1, length + 1),
        gradient_results[("tanh", length)],
        label=f"{length} Days"
    )

plt.title("Gradient Analysis Using Tanh Activation")
```

```
plt.xlabel("Time Step")
plt.ylabel("Gradient Norm")

plt.legend()
plt.grid(True)
plt.yscale("log")

plt.show()

# 11. Plot gradient norms for ReLU
plt.figure(figsize=(10, 5))

for length in sequence_lengths:

  plt.plot(
    range(1, length + 1),
    gradient_results[("relu", length)],
    label=f"{length} Days"
  )

plt.title("Gradient Analysis Using ReLU Activation")

plt.xlabel("Time Step")
plt.ylabel("Gradient Norm")

plt.legend()
plt.grid(True)
plt.yscale("log")

plt.show()
```

```
# 12. Compare activation functions
plt.figure(figsize=(10, 5))

for activation in activations:
    plt.plot(
        range(1, 101),
        gradient_results[(activation, 100)],
        label=activation
    )

plt.title("Tanh vs ReLU Gradient Comparison")

plt.xlabel("Time Step")
plt.ylabel("Gradient Norm")

plt.legend()
plt.grid(True)
plt.yscale("log")

plt.show()
```

7. Output

```
Stock Price Dataset:
        Date       Close
0 2023-01-01  100.993428
1 2023-01-02  100.808734
2 2023-01-03  103.420468
3 2023-01-04  106.122097
4 2023-01-05  103.430233

Activation: tanh
Sequence Length: 10
Loss: 0.0055321087
Initial Gradient Norm: 0.18370819
Final Gradient Norm: 0.19960952

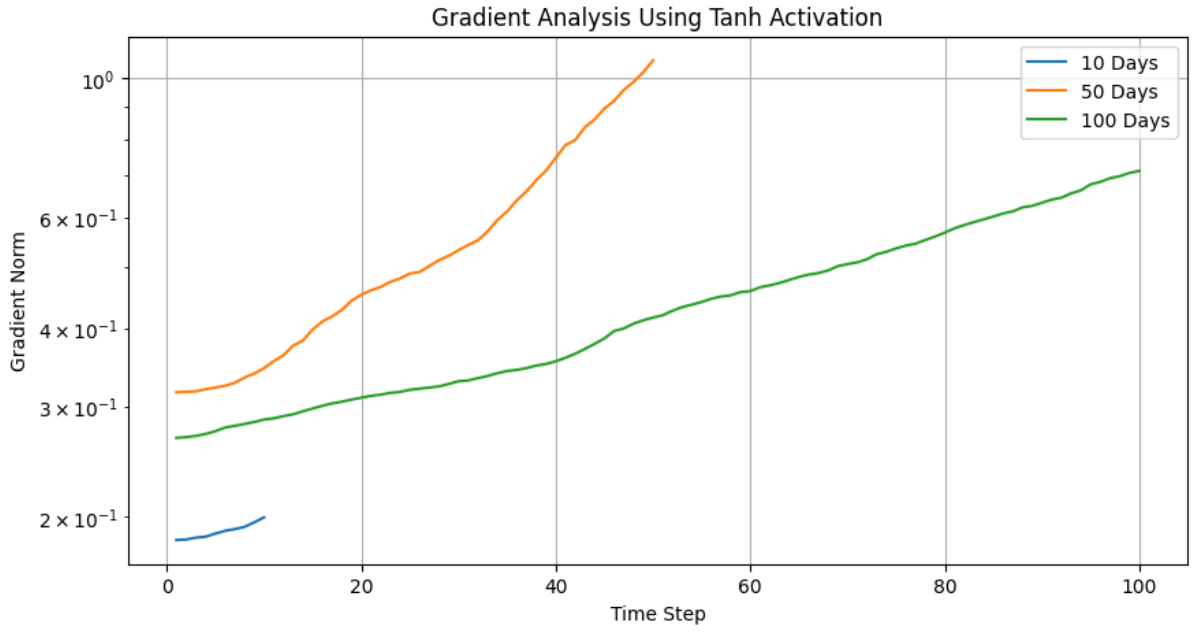
Activation: tanh
Sequence Length: 50
Loss: 0.13431258
Initial Gradient Norm: 0.31616375
Final Gradient Norm: 1.0687466

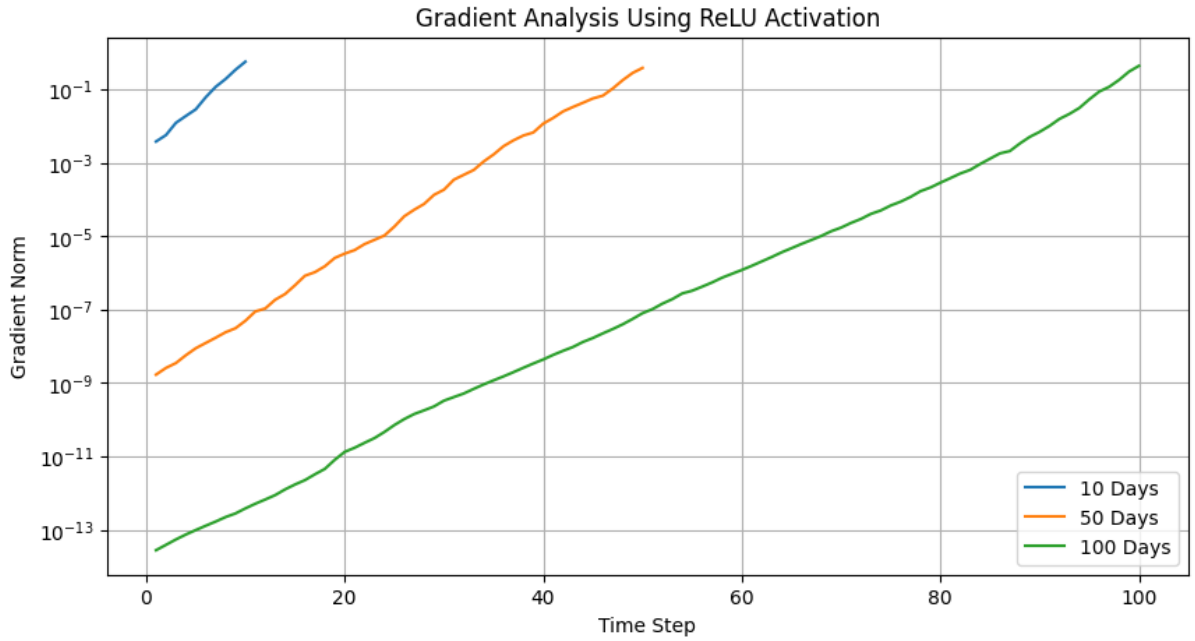
Activation: tanh
Sequence Length: 100
Loss: 0.06326951
Initial Gradient Norm: 0.267311
Final Gradient Norm: 0.7123634

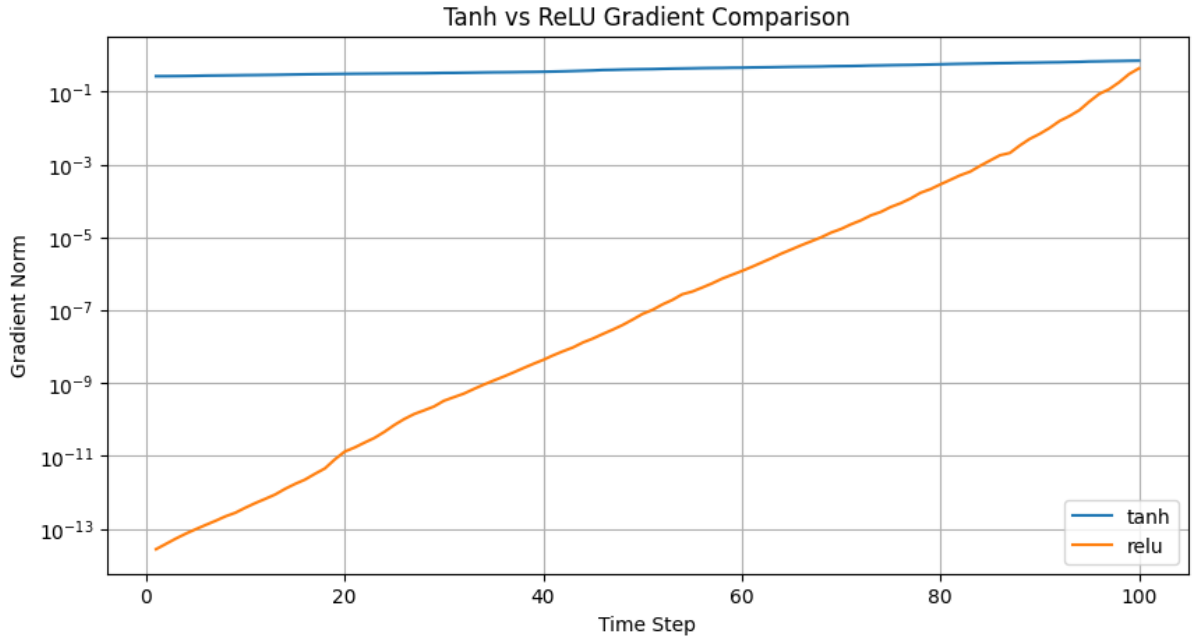
Activation: relu
Sequence Length: 10
Loss: 0.03885192
Initial Gradient Norm: 0.0038014366
Final Gradient Norm: 0.56914645

Activation: relu
Sequence Length: 50
Loss: 0.015483741
Initial Gradient Norm: 1.683061e-09
Final Gradient Norm: 0.38478446

Activation: relu
Sequence Length: 100
Loss: 0.021796335
Initial Gradient Norm: 2.7619237e-14
Final Gradient Norm: 0.43888474
```







8. Result

Thus, Backpropagation Through Time was analysed using a SimpleRNN cell and TensorFlow GradientTape. Gradient norms were calculated for sequence lengths of 10, 50 and 100 time steps using tanh and ReLU activation functions. The gradient plots were used to study vanishing and exploding gradient behaviour.